



**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1**



BÀI TẬP LẬP TRÌNH

Biên soạn : TS. NGUYỄN DUY PHƯƠNG

Hà Nội, tháng 12/2016

MỤC LỤC

I.	Lập trình với các cấu trúc dữ liệu cơ bản.....	4
1.1.	Các bài tập với dữ liệu kiểu số nguyên	4
1.2.	Các bài tập về mảng và ma trận	15
1.3.	Các bài tập về chuỗi ký tự	33
1.4.	Các bài tập về file và cấu trúc	38
II.	Lập trình dựa vào kỹ thuật duyệt và đệ quy	44
1.5.	Kỹ thuật vét cạn.....	44
1.6.	Kỹ thuật sinh kế tiếp	54
1.7.	Kỹ thuật quay lui	62
1.8.	Kỹ thuật nhánh cận.....	73
1.9.	Kỹ thuật quy hoạch động	75
III.	Lập trình dựa vào ngăn xếp, hàng đợi	80
1.10.	Kỹ thuật xử lý trên ngăn xếp	80
1.11.	Kỹ thuật xử lý trên hàng đợi	89
1.12.	Kỹ thuật xử lý trên danh sách liên kết.....	94
1.13.	Khử đệ quy dựa vào ngăn xếp và danh sách liên kết.....	102
IV.	Lập trình trên cây nhị phân	106
4.1.	Cây nhị phân.....	106
4.2.	Cây nhị phân tìm kiếm	108
4.3.	B-Cây (thuộc tìm kiếm bộ nhớ ngoài)	114
4.4.	Cây cân bằng	114
4.5.	Cây đồ đen.....	115
4.6.	Cây quyết định	116
4.7.	Cây mã tiền tố	116
V.	Lập trình trên Đồ thị	117
5.1.	Biểu diễn đồ thị	117
5.2.	Kỹ thuật DFS.....	119

5.3.	Kỹ thuật BFS	123
5.4.	Đồ thị Euler	136
5.5.	Đồ thị Hamilton.....	145
5.6.	Cây khung đồ thị	145
5.7.	Bài toán tìm đường đi ngắn nhất.....	155
5.8.	Bài toán luồng cực đại trên mạng.....	158
5.9.	Đồ thị hai phía	161
VI.	Các kỹ thuật sắp xếp và tìm kiếm	163
6.1.	Các phương pháp sắp xếp cơ bản.....	163
6.2.	Quicksort	169
6.3.	Heapsort	171
6.4.	Mergesort	172
6.5.	Sắp xếp bằng cơ số.....	173
6.6.	Các phương pháp tìm kiếm cơ bản	174
6.7.	Phép băm.....	177
6.8.	Tìm kiếm dựa vào cơ số	178
	TÀI LIỆU THAM KHẢO	180

I. Lập trình với các cấu trúc dữ liệu cơ bản

1.1. Các bài tập với dữ liệu kiểu số nguyên

BÀI 1.1.1: TỔNG CHỮ SỐ

Viết chương trình tính tổng chữ số của một số không quá 9 chữ số.

Dữ liệu vào:

Dòng đầu tiên ghi số bộ test.

Mỗi bộ test viết trên một dòng số nguyên tương ứng

Kết quả: Ghi ra màn hình

Mỗi bộ test viết ra trên một dòng giá trị tổng chữ số tương ứng

Ví dụ:

Input	Output
2	10
1234	2
1000001	

BÀI 1.1.2: BẮT ĐẦU VÀ KẾT THÚC

Viết chương trình kiểm tra một số nguyên dương bất kỳ (2 chữ số trở lên, không quá 9 chữ số) có chữ số bắt đầu và kết thúc bằng nhau hay không.

Dữ liệu vào:

Dòng đầu tiên ghi số bộ test.

Mỗi bộ test viết trên một dòng số nguyên dương tương ứng cần kiểm tra

Kết quả: Ghi ra màn hình

Mỗi bộ test viết ra YES hoặc NO, tương ứng với bộ dữ liệu vào

Ví dụ:

Input	Output
2	YES
12451	NO
1000012	

BÀI 1.1.3: BỘI SỐ CHUNG NHỎ NHẤT

Viết chương trình tính bội số chung nhỏ nhất của hai số nguyên dương không quá 7 chữ số.

Dữ liệu vào:

Dòng đầu tiên ghi số bộ test.

Mỗi bộ test viết trên một dòng hai số nguyên dương tương ứng, cách nhau một khoảng trống

Kết quả: Ghi ra màn hình

Mỗi bộ test viết ra trên một dòng giá trị bội số chung nhỏ nhất của hai số đó

Ví dụ

Input	Output
2	60
30 20	98754568
222222 8888888	

BÀI 1.1.4: SỐ CÓ TỔNG CHỮ SỐ CHIA HẾT CHO 10

Viết chương trình kiểm tra một số có thỏa mãn tính chất tổng chữ số của nó chia hết cho 10 hay không.

Dữ liệu vào:

Dòng đầu tiên ghi số bộ test. Mỗi bộ test viết trên một dòng một số nguyên dương, ít nhất 2 chữ số nhưng không quá 9 chữ số.

Kết quả:

Mỗi bộ test viết ra YES hoặc NO tùy thuộc kết quả kiểm tra.

Ví dụ

Input	Output
3	NO
3333	YES
555555	YES
123455	

BÀI 1.1.5: SỐ ĐẸP 1

Một số được coi là đẹp nếu nó là số thuận nghịch, tổng chữ số là số nguyên tố và tất cả các chữ số đều lẻ. Bài toán đặt ra là đếm xem trong một đoạn giữa hai số nguyên cho trước có bao nhiêu số đẹp như vậy.

Dữ liệu vào:

Dòng đầu tiên ghi số bộ test. Mỗi bộ test viết trên một dòng hai số nguyên dương tương ứng, cách nhau một khoảng trống. Các số đều không vượt quá 9 chữ số.

Kết quả:

Với mỗi bộ test viết ra số lượng các số thuận nguyên tố tương ứng.

Ví dụ

Input	Output
3	4
23 199	0
2345 6789	311
222222 99999999	

BÀI 1.1.6: SỐ ĐẸP 2

Một số được coi là đẹp nếu nó có tính chất thuận nghịch và tổng chữ số chia hết cho 10. Bài toán đặt ra là cho trước số chữ số. Hãy đếm xem có bao nhiêu số đẹp với số chữ số như vậy.

Dữ liệu vào:

Dòng đầu tiên ghi số bộ test.

Mỗi bộ test viết trên một dòng số chữ số tương ứng cần kiểm tra (lớn hơn 1 và nhỏ hơn 10)

Kết quả: Ghi ra màn hình

Mỗi bộ test viết ra số lượng số đẹp tương ứng

Ví dụ

Input	Output
2	1
2	90
5	

BÀI 1.1.7: TRÒ CHƠI ĐOÁN SỐ

Trong lúc rảnh rỗi, hai bạn sinh viên quyết định chơi trò đoán số giống học sinh cấp 1. Mỗi bạn nghĩ ra hai con số nguyên không âm sau đó viết ra tổng và hiệu của chúng (cũng là các số nguyên không âm). Công việc của bạn kia là xác định hai con số ban đầu. Ở một số lượt chơi, một bạn có thể cố tình đưa ra một cặp giá trị không thể là tổng và hiệu của hai số nguyên nào cả.

Viết chương trình giúp tính toán nhanh ra kết quả cho bài toán trên.

Dữ liệu vào:

Dòng đầu là số bộ test, không quá 200.

Mỗi dòng sau chứa hai số nguyên không âm s và d lần lượt là giá trị tổng và hiệu hai số. Cả hai số s và d đều không quá 10^4 .

Kết quả: Ghi ra màn hình

Với mỗi bộ dữ liệu, đưa ra hai số ban đầu, số lớn viết trước, cách nhau một khoảng trống. Nếu không thể có cặp số như vậy thì in ra “impossible”

Ví dụ

Input	Output
-------	--------

2	30 10
40 20	impossible
20 40	

BÀI 1.1.8: MÁY BÁN HÀNG TỰ ĐỘNG

Khi mua hàng bằng máy bán hàng tự động, người mua sẽ trả bằng một số tiền chẵn lớn hơn hoặc bằng giá của sản phẩm. Máy sẽ tính toán để trả lại số tiền thừa cho người mua. Giả sử trong máy chỉ có ba mệnh giá tiền là 1 dollar, 5 dollar và 10 dollar với quy ước mỗi lần trả chỉ được phép dùng ít hơn 5 tờ 1 dollar và ít hơn 2 tờ 5 dollar.

Hãy viết chương trình tính số tiền mỗi loại mà máy bán hàng tự động phải trả lại cho người mua.

Dữ liệu vào:

Dòng đầu tiên là số bộ test, mỗi bộ test ghi trên một dòng hai số nguyên không âm là giá của sản phẩm và tổng số tiền người mua đưa vào. Cả hai giá trị này đều không vượt quá 10^5 .

Kết quả:

Với mỗi bộ test, viết ra biểu diễn số tiền cần trả của máy bán hàng tự động theo mẫu trong bộ test ví dụ dưới đây. (Chú ý: giữa các số và các dấu luôn có đúng một khoảng trống, cả với dấu =, dấu * hoặc dấu +)

Ví dụ cho Input và Output:

<u>Input</u>	<u>Output</u>
3	$28 = 2 * 10 + 1 * 5 + 3 * 1$
72 100	$163 = 16 * 10 + 0 * 5 + 3 * 1$
37 200	$45 = 4 * 10 + 1 * 5 + 0 * 1$
5 50	

BÀI 1.1.9: (*) BIỂU DIỄN SỐ BẰNG QUE DIÊM

Một trong những cách biểu diễn số khá phổ biến trong các đồng hồ điện tử là sử dụng que diêm. Các ký tự số sẽ được biểu diễn như sau:

1 2 3 4 5 6 7 8 9 0

Với một số lượng que diêm cho trước, hãy các định số nhỏ nhất và số lớn nhất mà bạn có thể biểu diễn được.

Chú ý:

- Bạn không được phép để thừa que diêm nào khi xếp.
- Các số biểu diễn không được bắt đầu bằng số 0

Dữ liệu vào:

Dòng đầu tiên ghi số bộ test, không lớn hơn 100. Mỗi bộ test viết trên một dòng một số nguyên duy nhất không lớn hơn 100 là số que diêm bạn có.

Kết quả:

Với mỗi bộ test, output đưa ra hai số nguyên theo thứ tự là số nhỏ nhất và số lớn nhất có thể biểu diễn bởi số que diêm cho bởi input (mỗi số cách nhau một khoảng trống).

Ví dụ cho Input và Output:

Input	Output
4	7 7
3	6 111
6	8 711
7	108 7111111
15	

BÀI 1.1.10: ĐẾM SỐ CHÍNH PHƯƠNG TRONG ĐOẠN

Viết chương trình đếm trong một đoạn giữa hai số nguyên có bao nhiêu số chính phương.

Dữ liệu vào:

Dòng đầu tiên ghi số bộ test.

Mỗi bộ test viết trên một dòng hai số nguyên dương tương ứng, cách nhau một khoảng trống. Các số đều không quá 9 chữ số.

Kết quả: Ghi ra màn hình

Mỗi bộ test viết ra trên một dòng giá trị số các số chính phương đếm được.

Ví dụ:

Input	Output
2	10
23 199	34
2345 6789	

BÀI 1.1.11: SỐ THUẦN NGUYÊN TỔ

Một số được coi là thuần nguyên tố nếu nó là số nguyên tố, tất cả các chữ số là nguyên tố và tổng chữ số của nó cũng là một số nguyên tố. Bài toán đặt ra là đếm xem trong một đoạn giữa hai số nguyên cho trước có bao nhiêu số thuần nguyên tố.

Dữ liệu vào:

Dòng đầu tiên ghi số bộ test.

Mỗi bộ test viết trên một dòng hai số nguyên dương tương ứng, cách nhau một khoảng trống. Các số đều không vượt quá 9 chữ số.

Kết quả: Ghi ra màn hình

Mỗi bộ test viết ra số lượng các số thuần nguyên tố tương ứng.

Ví dụ

Input	Ouput
-------	-------

2	1
23 199	15
2345 6789	

BÀI 1.1.12: SỐ CÓ TỔNG CHỮ SỐ CHIA HẾT CHO 10

Viết chương trình kiểm tra một số có thỏa mãn tính chất tổng chữ số của nó chia hết cho 10 hay không.

Dữ liệu vào:

Dòng đầu tiên ghi số bộ test. Mỗi bộ test viết trên một dòng một số nguyên dương, ít nhất 2 chữ số nhưng không quá 9 chữ số.

Kết quả:

Mỗi bộ test viết ra YES hoặc NO tùy thuộc kết quả kiểm tra.

Ví dụ:

Input	Output
3	NO
3333	YES
555555	YES
123455	

BÀI 1.1.13: SỐ ĐẸP 7

Một số được coi là đẹp nếu nó là số thuận nghịch, tổng chữ số là số nguyên tố và tất cả các chữ số đều lẻ. Bài toán đặt ra là đếm xem trong một đoạn giữa hai số nguyên cho trước có bao nhiêu số đẹp như vậy.

Dữ liệu vào:

Dòng đầu tiên ghi số bộ test. Mỗi bộ test viết trên một dòng hai số nguyên dương tương ứng, cách nhau một khoảng trống. Các số đều không vượt quá 9 chữ số.

Kết quả:

Với mỗi bộ test viết ra số lượng các số thuần nguyên tố tương ứng.

Ví dụ

Input	Output
3	4
23 199	0
2345 6789	311
222222 99999999	

BÀI 1.1.14:

Hãy viết chương trình tìm số các số tự nhiên N thỏa mãn đồng thời những điều kiện dưới đây ($N \leq 2^{31}$):

- N là số có K chữ số ($K \leq 15$).
- N là số nguyên tố.
- Đảo ngược các chữ số của N cũng là một số nguyên tố.
- Tổng các chữ số của N cũng là một số nguyên tố.
- Mỗi chữ số của N cũng là các số nguyên tố ;
- Thời gian thực hiện chương trình không quá 1sec.

Dữ liệu vào (Input) cho bởi file data.in theo khuôn dạng:

- Dòng đầu tiên ghi lại số tự nhiên M là số lượng các test ($M \leq 100$).
- M dòng kế tiếp ghi lại mỗi dòng một test. Mỗi test bao gồm một số K . Hai số được viết cách nhau một vài khoảng trống.

Kết quả ra (Output): ghi lại M dòng trong file ketqua.out, mỗi dòng ghi lại bộ hai số N, X . Trong đó X là số các số có N chữ số thỏa mãn yêu cầu của bài toán. Ví dụ dưới đây minh họa cho file input và output của bài toán.

<u>Input.in</u>	<u>Output.out</u>	
5	2	0
2	3	8
3	4	15
4	5	46
5	7	359
7		

BÀI 1.1.15:

Trong ngày đầu tiên phát hành các số điện thoại di động, công ty viễn thông dự định khuyến mại cho N khách hàng đăng ký trước nhất các số điện thoại loại 1, M khách hàng kế tiếp số điện thoại loại 2 và K khách hàng cuối cùng các số điện thoại loại 3. Các số điện thoại loại 1, loại 2 và loại 3 có tính chất sau:

- Số loại 3 (Loại 3): là các số điện thoại mà sáu số cuối cùng của nó tạo thành một số thuận nghịch có sáu chữ số. Ví dụ số : 0913.104401.
- Số loại 2 (Loại 2): là các số điện thoại Loại 3 có tổng sáu số cuối cùng của nó là một số chia hết cho 10. Ví dụ số : 0913.104401.
- Số loại 1 (Loại 1): là các số điện thoại Loại 2 có tổng sáu số cuối cùng của nó không chứa bất kỳ số 0 nào. Ví dụ số : 0913.686686.

Bài toán được đặt ra là cho trước một phương án N, M, K, hãy trả lời “YES” nếu công ty thực hiện được, trả lời “NO” nếu công ty không thực hiện được.

Input: Dòng đầu tiên ghi số bộ test, không lớn hơn 100. Mỗi bộ test là một bộ 3 số N, M, K được ghi trên một dòng. Các số được ghi cách nhau một vài khoảng trống.

Output: Với mỗi bộ test, viết ra trên một dòng giá trị “YES” hoặc “NO” tương ứng với phương án thực hiện được, hoặc phương án không thực hiện được.

Ví dụ cho Input và Output:

INPUT	OUTPUT
5	NO
100 100 200	NO
50 150 200	YES
100 50 300 120	YES
50 500	NO
140 50 700	

BÀI 1.1.16:

Số N nguyên hệ cơ số ACM là những số nguyên thông thường sử dụng các ký hiệu từ 0, 1, ..., 9 làm ký hiệu của hệ đếm (Ví dụ số 719_{ACM}). Nguyên tắc chung để đổi một số $A = (a_1, a_2, \dots, a_N)$ ở hệ cơ số ACM sang số ở hệ cơ số 10 được thực hiện như sau:

$$K_{10} = \sum_{i=1}^N a_i * (i!), \text{ trong đó } a_i \text{ chữ số tại vị trí thứ } i \text{ của ở hệ cơ số ACM.}$$

Ví dụ:

$$A = 719_{ACM} = 9.(1!) + 1.(2!) + 7.(3!) = 53_{10}.$$

Nhiệm vụ của bạn là viết một chương trình đọc một số nguyên ở hệ cơ số ACM rồi đổi số đó thành số hệ cơ số 10.

Dữ liệu vào:

Dữ liệu vào gồm nhiều bộ dữ liệu tương ứng với nhiều test. Dòng đầu tiên chứa một số nguyên không lớn hơn 100 là số lượng các bộ dữ liệu. Những dòng tiếp theo chứa các bộ dữ liệu. Mỗi bộ dữ liệu được viết trên một dòng. Mỗi dòng viết một số nhỏ hơn 2^{32} là số ở hệ cơ số ACM.

Dữ liệu ra:

Với mỗi bộ dữ liệu, ghi ra trên một dòng một số được chuyển đổi.

Ví dụ dữ liệu vào	Ví dụ dữ liệu ra
6	53
719	1
1	7
15	8
110	8
102	0
8	

BÀI 1.1.17: SỐ MAY MẮN

Hoàng yêu thích các số may mắn. Ta biết rằng một số là số may mắn nếu biểu diễn thập phân của nó chỉ chứa các chữ số may mắn là 4 và 7. Ví dụ, các số 47, 744, 4 là số may mắn và 5, 17, 467 không phải. Hoàng muốn tìm số may mắn bé nhất có tổng các chữ số bằng n . Hãy giúp anh ấy

Dữ liệu vào: Dòng đầu ghi số bộ test, mỗi bộ test có một dòng chứa số nguyên n ($1 \leq n \leq 106$) — tổng các chữ số của số may mắn cần tìm.

Kết quả: In ra trên 1 dòng số may mắn bé nhất, mà tổng các chữ số bằng n . Nếu không tồn tại số thỏa mãn, in ra -1.

Ví dụ

Input	Output
2	47
11	-1
10	

1.2. Các bài tập về mảng và ma trận

BÀI 1.2.1: SỐ CẤP BẰNG NHAU TRONG DÃY

Viết chương trình đếm các cặp số bằng nhau liên tiếp trong dãy số nguyên.

Dữ liệu vào:

Dòng đầu tiên ghi số bộ test.

Mỗi bộ test có hai dòng:

- Dòng đầu ghi số phần tử của dãy, không quá 30
- Dòng tiếp theo ghi các phần tử của dãy, mỗi phần tử cách nhau một khoảng trống. Các phần tử không quá 100.

Kết quả: Ghi ra màn hình

Mỗi bộ test viết ra trên một dòng giá trị tổng chữ số tương ứng

Ví dụ:

Input	Output
2	1
4	6
1 3 3 4	
12	
1 2 3 3 3 3 4 4 5 5 5 1	

BÀI 1.2.2: ĐẾM CÁC SỐ LỚN HƠN SỐ ĐỨNG TRƯỚC TRONG DÃY

Cho một dãy số nguyên dương có n phần tử ($2 \leq n \leq 50$). Hãy liệt kê số các phần tử trong dãy không nhỏ hơn các số đứng trước nó (tính cả phần tử đầu tiên).

Dữ liệu vào:

- Dòng 1 ghi số bộ test

- Với mỗi bộ test: dòng đầu tiên ghi số n. Dòng tiếp theo ghi n số nguyên dương của dãy (các số không vượt quá 1000).

Kết quả:

Ghi ra màn hình trên một dòng số các phần tử thỏa mãn.

Ví dụ:

<u>Input</u>	<u>Output</u>
2	5
7	3
3 5 6 8 4 2 9	
15	
9 8 123 7 11 14 18 21 399 10 5 4 1 2 3	

BÀI 1.2.3: ĐOÁN SỐ TIẾP THEO

An và Bình chơi trò chơi số học đơn giản. Dãy số mà An đưa ra là $A = \{1, 1, 3, 4, 5, 9, 7, 16, 9, \dots\}$ và đồ Bình tìm ra số tiếp theo trong dãy đó. Rất nhanh chóng, Bình tìm được số tiếp theo là số 25. Bình đố lại An, nếu cho trước một số k không quá 100, hãy tính số đứng vị trí đó trong dãy đã cho (thứ tự trên dãy tính từ 1). Bạn hãy giúp An tính ra kết quả trên.

Dữ liệu vào:

Dòng đầu là số bộ test, không quá 20. Mỗi bộ test ghi trên một dòng số nguyên dương k.

Kết quả:

Với mỗi bộ test, đưa ra trên một dòng giá trị ở vị trí k của dãy.

Ví dụ:

<u>C.IN</u>	<u>Kết quả</u>
3	1
1	4
4	25
10	

BÀI 1.2.4: TỔNG HAI ĐA THỨC

Cho hai đa thức $P(x)$ -bậc n và $Q(x)$ -bậc m có các hệ số nguyên, n và m không quá 100. Hãy viết chương trình tính tổng hai đa thức trên.

Dữ liệu vào:

Dòng đầu tiên chứa số nguyên N là số bộ dữ liệu ($1 \leq N \leq 10$).

Mỗi bộ dữ liệu gồm 4 dòng:

- Dòng 1 ghi số n là bậc của P . Dòng 2 ghi $n+1$ số nguyên tương ứng là hệ số của P từ P_0 đến P_n
- Dòng 3 ghi số m là bậc của Q . Dòng 4 ghi $m+1$ số nguyên tương ứng là hệ số của Q , từ Q_0 đến Q_m

Kết quả: Ghi ra màn hình

Với mỗi bộ dữ liệu vào, in ra kết quả trên hai dòng: Dòng 1 ghi bậc của đa thức tổng. Dòng 2 ghi lần lượt các hệ số của đa thức tổng, tính từ 0.

Ví dụ:

D.IN	Kết quả
1	5
3	2 3 5 2 3 3
1 2 3 4	
5	
1 1 2 -2 3 3	

BÀI 1.2.5: TÌM CÁC VỊ TRÍ BẰNG NHAU CỦA HAI MA TRẬN

Cho hai ma trận vuông A và B chỉ gồm số nguyên dương cấp n . Hãy viết chương trình tìm ra các vị trí bằng nhau trong hai ma trận (vị trí $[i,j]$ được coi là bằng nhau nếu $A[i,j]=B[i,j]$).

Dữ liệu vào:

Dòng đầu tiên ghi số bộ test. Với mỗi bộ test: Dòng đầu tiên ghi số n ; n dòng tiếp theo ghi ma trận A ; n dòng tiếp theo ghi ma trận B

Kết quả (ghi ra màn hình):

Với mỗi bộ test ghi ra một số nguyên dương m là số vị trí bằng nhau. m dòng tiếp theo ghi hai giá trị chỉ số của từng cặp (cách nhau một khoảng trống). (Chú ý: các chỉ số đều tính từ 0 đến $n-1$).

Ví dụ:

Input	Output
1	2
2	0 1
1 1	1 1
1 2	
9 1	
4 2	

BÀI 1.2.6: SỐ FIBONACI THỨ N

Dãy Fibonacci được định nghĩa như sau: $F(0) = 0$, $F(1)=1$, ..., $F(n)=F(n-1)+F(n-2)$. Cho trước số nguyên dương n (không quá 45), hãy tính số Fibonacci thứ n .

Dữ liệu vào:

Dòng 1 ghi số bộ test. Mỗi bộ test ghi trên một dòng số nguyên dương n .

Kết quả (ghi ra màn hình):

Với mỗi bộ test ghi ra giá trị số Fibonacci thứ n .

Ví dụ:

Input	Output
3	1
1	13
7	55
10	

BÀI 1.2.7: TÍCH MA TRẬN VỚI CHUYỂN VỊ CỦA NÓ

Cho ma trận A chỉ gồm các số nguyên dương cấp $n \times m$. Hãy viết chương trình tính tích của A với ma trận chuyển vị của A.

Dữ liệu vào:

Dòng đầu tiên ghi số bộ test. Với mỗi bộ test: Dòng đầu tiên ghi hai số n và m là bậc của ma trận a; n dòng tiếp theo, mỗi dòng ghi m số của một dòng trong ma trận A.

Kết quả (ghi ra màn hình):

Với mỗi bộ test ghi ra ma trận tích tương ứng, mỗi số cách nhau đúng một khoảng trống.

Ví dụ:

Input	Output
1	5 11
2 2	11 25
1 2	
3 4	

BÀI 1.2.8: SỐ XUẤT HIỆN NHIỀU LẦN NHẤT TRONG DÃY

Cho một dãy số nguyên dương không quá 100 phần tử, các giá trị trong dãy không quá 30000. Hãy xác định xem số nào là số xuất hiện nhiều lần nhất trong dãy. Chú ý: trong trường hợp nhiều số khác nhau cùng xuất hiện số lần bằng nhau và là lớn nhất thì in ra tất cả các số đó theo thứ tự xuất hiện trong dãy ban đầu.

Dữ liệu vào:

Dòng đầu là số bộ test, không quá 20.

Mỗi bộ test gồm hai dòng. Dòng đầu ghi số phần tử của dãy, dòng tiếp theo ghi các phần tử của dãy.

Kết quả: Ghi ra màn hình

Với mỗi bộ test, đưa ra số xuất hiện nhiều lần nhất trong dãy đã cho.

Ví dụ:

Input	Output
2	1
10	1 2 3 4 5 6 7 8 9 0
1 2 3 1 2 3 1 2 3 1	
10	
1 2 3 4 5 6 7 8 9 0	

BÀI 1.2.9: MA TRẬN ĐƠN VỊ

Một ma trận vuông A cấp n chỉ gồm các số nguyên dương. A được gọi là ma trận đơn vị nếu tất cả các phần tử trong A đều là 0 hoặc 1. Viết chương trình kiểm tra xem một ma trận có đối xứng hay không.

Dữ liệu vào:

Dòng đầu tiên ghi số bộ test (không quá 10). Với mỗi bộ test: Dòng đầu tiên ghi số n là bậc của ma trận A; n dòng tiếp theo, mỗi dòng ghi n số của một dòng trong ma trận A.

Các giá trị đều không quá 100.

Kết quả (ghi ra màn hình):

Với mỗi bộ test ghi ra màn hình chữ YES nếu đó đúng là ma trận đơn vị, NO nếu ngược lại.

Ví dụ:

Input	Output
2	NO
2	YES
1 1	
1 2	
4	
1 1 0 0	
1 0 0 1	
0 0 1 1	
1 0 1 0	

BÀI 1.2.10: TỔNG TAM GIÁC

Số tam giác thứ n là tổng của n số tự nhiên đầu tiên, $T(n) = 1 + \dots + n$. Nó cũng chính là số lượng điểm trong một mảng tam giác có cạnh là n , ví dụ như với $T(4)$ thì:

```

      X
     X X
    X X X
   X X X X
    
```

Hãy viết chương trình tính tổng có trọng số của số tam giác:

$$W(n) = \sum_{k=1}^n k \cdot T(k+1)$$

Dữ liệu vào:

Dòng đầu tiên chứa số nguyên N là số bộ dữ liệu ($1 \leq N \leq 1000$).

Mỗi bộ dữ liệu gồm 1 dòng chứa duy nhất 1 số nguyên n ($1 \leq n \leq 300$) là số điểm trên cạnh của tam giác.

Kết quả: Ghi ra màn hình

Với mỗi bộ dữ liệu vào, in ra trên một dòng số thứ tự của bộ dữ liệu (1 đến N), một dấu cách, giá trị của n trong bộ dữ liệu, một dấu cách và tổng có trọng số $W(n)$ của những số tam giác tương ứng với n .

Ví dụ:

D.IN	Kết quả
4	1 3 45
3	2 4 105
4	3 5 210
5	4 10 2145
10	

BÀI 1.2.11: TRỘN HAI DÃY VÀ SẮP XẾP

Cho hai dãy số nguyên dương A và B không quá 100 phần tử, các giá trị trong dãy không quá 30000 và số phần tử của hai dãy bằng nhau. Hãy trộn hai dãy với nhau sao cho dãy A

được đưa vào các vị trí có chỉ số chẵn, dãy B được đưa vào các vị trí có chỉ số lẻ. Đồng thời, dãy A được sắp xếp tăng dần, còn dãy B được sắp xếp giảm dần. (Chú ý: chỉ số tính từ 0)

Dữ liệu vào

- Dòng 1 ghi số bộ test
- Với mỗi bộ test: dòng đầu tiên ghi số n. Dòng tiếp theo ghi n số nguyên dương của dãy A. Dòng tiếp theo ghi n số nguyên dương của dãy B

Kết quả

Với mỗi bộ test, đưa ra thứ tự bộ test và dãy kết quả.

Ví dụ:

<u>Input</u>	<u>Output</u>
2	Test 1:
5	1 3 1 3 2 2 2 1 3 1
1 2 3 1 2	Test 2:
3 1 2 3 1	1 8 2 6 4 5 7 2
4	
4 2 7 1	
5 6 2 8	

BÀI 1.2.12: MA TRẬN XOÁY ỐC NGƯỢC

Ma trận xoáy ốc ngược cấp N là ma trận vuông có $N \times N$ phần tử. Các số được điền vào ma trận theo chiều kim đồng hồ theo thứ tự giảm dần. Hãy xây dựng ma trận xoáy ốc ngược sao cho các số trong ma trận đều là số nguyên dương chia hết cho 3 (tính cả số 0).

Dữ liệu vào

- Dòng 1 ghi số bộ test
- Mỗi bộ test ghi số N ($1 < N < 20$).

Kết quả

Ghi ra thứ tự bộ test và ma trận xoáy ốc ngược tương ứng

Ví dụ:

<u>Input</u>	<u>Output</u>
1 3	Test 1: 24 21 18 3 0 15 6 9 12

BÀI 1.2.13: ĐẾM CÁC SỐ NGUYÊN TỐ TRONG DÃY

Cho dãy số A có n phần tử chỉ bao gồm các số nguyên dương (không quá 9 chữ số). Hãy xác định các số nguyên tố trong dãy và đếm xem mỗi số xuất hiện bao nhiêu lần.

Dữ liệu vào

Dòng đầu tiên ghi số bộ test. Với mỗi bộ test: dòng đầu ghi số n; dòng tiếp theo ghi n số của dãy,

Kết quả

Với mỗi bộ test ghi ra thứ tự bộ test, sau đó lần lượt là các số nguyên tố trong dãy theo thứ tự từ nhỏ đến lớn và số lần xuất hiện của nó.

Ví dụ

<u>Input</u>	<u>Output</u>
2 10 1 2 3 3 2 1 3 2 7 8 8 10 7 6 7 3 2 5 7	Test 1: 2 xuất hiện 3 lần 3 xuất hiện 3 lần 7 xuất hiện 1 lần Test 2: 2 xuất hiện 1 lần 3 xuất hiện 1 lần 5 xuất hiện 1 lần 7 xuất hiện 2 lần

BÀI 1.2.14: TÍCH MA TRẬN VỚI CHUYỂN VỊ CỦA NÓ

Cho một số nguyên dương N không quá 20. Ma trận vuông A cấp $N \times N$ được tạo theo mẫu trong bảng dưới. Viết chương trình tính tích của A với chuyển vị của A.

Với N = 4	Với N = 5
1 0 0 0	1 0 0 0 0
1 2 0 0	1 2 0 0 0
1 2 3 0	1 2 3 0 0
1 2 3 4	1 2 3 4 0
	1 2 3 4 5

Dữ liệu vào:

Dòng 1 ghi số bộ test. Mỗi bộ test ghi trên một dòng số N ($1 < N < 20$).

Kết quả:

Ghi thứ tự bộ test, sau đó là N hàng ghi ma trận kết quả. Tiếp theo là một dòng trống.

Ví dụ

Input	Output
1	Test 1:
4	1 1 1 1
	1 5 5 5
	1 5 14 14
	1 5 14 30

BÀI 1.2.15: ĐỖ XE TỐI ƯU

Khi mua sắm trên khu Long Street, Michael thường đỗ xe của mình ở một số vị trí ngẫu nhiên và đi bộ vào cửa hàng. Bạn hãy giúp Michael chọn một chỗ đỗ xe để khoảng cách phải đi bộ khi mua hàng là nhỏ nhất.

Long Street có thể coi như là một đường thẳng mà tất cả những điểm mua hàng là các điểm có tọa độ nguyên.

Dữ liệu vào:

Dòng đầu tiên chứa một số nguyên $1 \leq t \leq 100$ là số lượng bộ test. Mỗi bộ test gồm 2 dòng, dòng đầu tiên ghi số cửa hàng n mà Michael muốn qua mua hàng, $1 \leq n \leq 20$ và dòng thứ hai ghi n số nguyên là tọa độ các điểm này trên phố Long Street, $0 \leq x_i \leq 99$.

Kết quả:

Với mỗi bộ test, in ra màn hình trên một dòng khoảng cách nhỏ nhất phải đi bộ với chỗ đỗ xe tối ưu.

Ví dụ cho Input và Output:

Input	Ouput
2	152
4	70
24 13 89 37	
6	
7 30 41 14 39 42	

BÀI 1.2.16:

Ta định nghĩa một từ là dãy các kí tự không chứa khoảng trống (space), dấu tab, dấu xuống dòng ('\n'), dấu về đầu dòng ('\r') và dấu kết thúc dòng ('\0'). Tần xuất xuất hiện của từ W trong tập văn bản D_1 và D_2 , ký hiệu là $P(W)$ được tính theo công thức:

$$P(W) = \frac{N_1(W) + N_2(W)}{N(D_1) + N(D_2)}; \text{ trong đó } N_i(W) \text{ là số lần xuất hiện từ } W \text{ trong } D_i, N(D_i) \text{ là tổng}$$

số từ của tập văn bản D_i ($i=1, 2$).

Cho hai file văn bản data1.in và data2.in. Sử dụng CTDL Mảng, hãy tìm tập các từ và tần xuất xuất hiện của mỗi từ hoặc xuất hiện trong data1.in hoặc xuất hiện trong data2.in. Tập các từ tìm được ghi lại trong file Ketqua.out theo khuôn dạng:

- Dòng đầu tiên ghi lại số tự nhiên K là số từ W tìm được theo yêu cầu của bài toán.
- K dòng kế tiếp, mỗi dòng ghi lại một từ W và tần xuất xuất hiện $P(W)$ thỏa mãn yêu cầu của bài toán. W và $P(W)$ được viết cách nhau bởi một vài khoảng trống.

Ví dụ với file data1.in và data2.in dưới đây sẽ cho ta file Ketqua.out của bài toán.

data1.in	data2.in	Ketqua.out
AB AC AD AE AF	AB AC AD AH AK	3
AB AC AD AE AF	AB AC AD AH AK	AB 0.2
		AC 0.2
		AD 0.2

BÀI 1.2.17:

Ta định nghĩa một từ là dãy các kí tự không chứa khoảng trống (space), dấu tab, dấu xuống dòng ('\n'), dấu về đầu dòng ('\r') và dấu kết thúc dòng ('\0'). Tần suất xuất hiện của từ W trong tập văn bản D_1 và D_2 , ký hiệu là $P(W)$ được tính theo công thức:

$$P(W) = \frac{N_1(W) + N_2(W)}{N(D_1) + N(D_2)}; \text{ trong đó } N_i(W) \text{ là số lần xuất hiện từ } W \text{ trong } D_i, N(D_i) \text{ là tổng}$$

số từ của tập văn bản D_i ($i=1, 2$).

Cho hai file văn bản *data1.in* và *data2.in*. Sử dụng CTDL Mảng, hãy tìm tập các từ và tần suất xuất hiện của các từ xuất hiện trong file *data1.in* nhưng không xuất hiện trong file *data2.in*. Tập các từ tìm được ghi lại trong file *Ketqua.out* theo khuôn dạng:

- Dòng đầu tiên ghi lại số tự nhiên K là số từ W tìm được theo yêu cầu của bài toán.
- K dòng kế tiếp, mỗi dòng ghi lại một từ W và tần suất xuất hiện $P(W)$ thỏa mãn yêu cầu của bài toán. W và $P(W)$ được viết cách nhau bởi một vài khoảng trống.

Ví dụ với file *data1.in* và *data2.in* dưới đây sẽ cho ta file *Ketqua.out* của bài toán.

data1.in	data2.in	Ketqua.out
AB AC AD AE AF	AB AC AD AH AK	7
AB AC AD AE AF	AB AC AD AH AK	AB 0.2
		AC 0.2
		AD 0.2
		AE 0.1
		AF 0.1
		AH 0.1
		AK 0.1

BÀI 1.2.18:

Ta định nghĩa một từ là dãy các kí tự không chứa khoảng trống (space), dấu tab, dấu xuống dòng ('\n'), dấu về đầu dòng ('\r') và dấu kết thúc dòng ('\0'). Tần suất xuất hiện của từ W trong tập văn bản D_1 và D_2 , ký hiệu là $P(W)$ được tính theo công thức:

$$P(W) = \frac{N_1(W) + N_2(W)}{N(D_1) + N(D_2)}; \text{ trong đó } N_i(W) \text{ là số lần xuất hiện từ } W \text{ trong } D_i, N(D_i) \text{ là tổng}$$

số từ của tập văn bản D_i ($i=1, 2$).

Cho hai file văn bản data1.in và data2.in. *Sử dụng CTDL Mảng*, hãy tìm tập các từ và tần xuất xuất hiện của mỗi từ đồng thời trong cả hai tập data1.in và data2.in. Tập các từ tìm được ghi lại trong file Ketqua.out theo khuôn dạng:

- Dòng đầu tiên ghi lại số tự nhiên K là số từ W tìm được theo yêu cầu của bài toán.
- K dòng kế tiếp, mỗi dòng ghi lại một từ W và tần xuất xuất hiện $P(W)$ thỏa mãn yêu cầu của bài toán. W và $P(W)$ được viết cách nhau bởi một vài khoảng trống.

Ví dụ với file data1.in và data2.in dưới đây sẽ cho ta file Ketqua.out của bài toán.

data1.in	data2.in	Ketqua.out
AB AC AD AE AF	AB AC AD AH AK	2
AB AC AD AE AF	AB AC AD AH AK	AE 0.1
		AF 0.1

BÀI 1.2.19:

Cho tập các số tự nhiên ghi lại trong file data.in theo từng dòng; mỗi dòng ghi lại nhiều nhất 10 số; hai số được viết cách nhau một vài khoảng trống; mỗi số tự nhiên có thể xuất hiện nhiều lần ở những vị trí khác nhau trong file. Ta gọi tần xuất xuất hiện số tự nhiên X

trong file data.in là $P(X) = \frac{N(X)}{K}$; trong đó $N(X)$ là số lần xuất hiện số tự nhiên X trong

file data.in, K là số các số tự nhiên trong file data.in. *Sử dụng CTDL mảng*, hãy viết chương trình tìm số vừa nguyên tố vừa thuận nghịch X có $P(X)$ lớn nhất đầu tiên tìm được trong file data.in. Ví dụ với file data.in được cho dưới đây, ta tìm được số $X = 131$ vừa là số nguyên tố, vừa là số thuận nghịch với tần xuất xuất hiện lớn nhất là $P(X) = 3/30 = 0.1$.

data.in

```

10 131 171 13 37 27 30 23 77 444
10 131 171 20 23 77 23 27 77 444
10 131 171 20 23 77 23 27 77 444
    
```

BÀI 1.2.20:

Cho tập các số tự nhiên trong file data.in được ghi theo từng dòng, mỗi dòng ghi nhiều nhất 5 số, hai số được viết cách nhau một vài khoảng trống. Biết rằng, mỗi số tự nhiên trong file data.in hoặc là số nguyên tố, hoặc là số thuận nghịch và có thể xuất hiện nhiều lần ở những vị trí khác nhau trong file. *Sử dụng cấu trúc dữ liệu mảng*, hãy viết chương trình tách tập các số và đếm số lần xuất hiện của mỗi số trong file data.in thành 3 file ketqua1.out, ketqua2.out, ketqua3.out thỏa mãn những yêu cầu dưới đây.

- File ketqua1.out ghi lại các số nguyên tố nhưng không là số thuận nghịch cùng với số lần xuất hiện của nó trong file data.in;
- File ketqua2.out ghi lại các số thuận nghịch nhưng không là nguyên tố cùng với số lần xuất hiện của nó trong file data.in;
- File ketqua3.out ghi lại các số vừa là số nguyên tố vừa là số thuận nghịch cùng với số lần xuất hiện của nó trong file data.in;
- Khuôn dạng của các file kết quả được qui định như sau:
 - Dòng đầu tiên của mỗi file ghi lại số các số của mỗi file kết quả;
 - Những dòng kế tiếp mỗi dòng ghi lại một số cùng với số lần xuất hiện của nó trong file data.in. Hai số được viết cách nhau một vài khoảng trống.

Ví dụ dưới đây minh họa cho các file data.in, ketqua1.out, ketqua2.out và ketqua3.out.

Data.in	Ketqua1.out	Ketqua2.out	Ketqua3.out
10007 10009 10801 10901 13831	2	2	2
10007 10009 10801 10901 34543	10007 4	10801 4	13831 2
10007 10009 10801 10901 13831	10009 4	10901 4	34543 2
10007 10009 10801 10901 34543			

BÀI 1.2.21:

Cho một hình vuông gồm 5 x 5 hình vuông đơn vị như hình vẽ sau:

3	5	1	1	1
5	0	0	3	3
1	0	3	4	3
1	3	4	2	1
1	3	3	1	3

Hãy điền các chữ số từ 0 đến 9 vào mỗi hình vuông đơn vị sao cho những điều kiện sau được thỏa mãn:

- (i) Đọc từ trái sang phải theo hàng ta nhận được năm số nguyên tố có năm chữ số;
- (ii) Đọc từ trên xuống dưới theo cột ta nhận được năm số nguyên tố có năm chữ số;
- (iii) Đọc theo hai đường chéo chính từ trái sang phải ta nhận được hai số nguyên tố có năm chữ số;
- (iv) Tổng các chữ số trong mỗi số nguyên tố đều bằng S cho trước (trong ví dụ trên S=11).

Input: Dòng đầu tiên ghi số bộ test, không lớn hơn 100. Mỗi bộ test viết trên một dòng một bộ đôi hai số S, T tương ứng với tổng các chữ số trong mỗi số nguyên tố và giá trị ô vuông trên cùng góc trái.

Output: Với mỗi bộ test, output đưa ra một số duy nhất là số các lời giải của bài toán. Mỗi lời giải của bài toán là một hình vuông gồm 5×5 hình vuông đơn vị. Hai lời giải được xem là khác nhau nếu tồn tại một ô vuông đơn vị ở cùng một vị trí có giá trị khác nhau.

Ví dụ cho Input và Output của bài toán:

INPUT	OUTPUT
3	5
11 3	0
11 9	0
11 8	

BÀI 1.2.22:

Trò chơi Rubik trên mặt phẳng có thể được mô tả bằng dãy các số từ 1 đến 8 và được chia thành hai hàng. Ta qui ước trạng thái khởi đầu S của Rubik là trạng thái trong hình 1. Người ta đã chứng minh được rằng, từ trạng thái khởi đầu S ta có thể dịch chuyển thành trạng thái T bất kỳ trên các thao tác A, B, C dưới đây:

A : Đổi hàng trên cho hàng dưới (Hình 2).

B : Tạo nên một hoán vị vòng trên mỗi hàng (Hình 3).

C : Quay 90 độ 4 ô ở giữa (Hình 4)

Hình 1. Trạng thái khởi đầu S

1	2	3	4
8	7	6	5

Hình 2. Thao tác A				Hình 3. Thao tác B				Hình 4. Thao tác C			
8	7	6	5	4	1	2	3	1	7	2	4
1	2	3	4	5	8	7	6	8	6	3	5

Yêu cầu: Cho trước một trạng thái kết thúc T. Hãy chuyển S thành T sao cho số các phép A, B, C thực hiện là ít nhất.

Input: Dòng đầu tiên ghi số bộ test, không lớn hơn 100. Mỗi bộ test được tổ chức trên một dòng là giá trị trạng thái kết thúc T.

Output: Với mỗi bộ test, viết ra trên một dòng số các bước A, B, C ít nhất.

INPUT	OUTPUT
2	7
2 6 8 4 5 7 3 1	0
1 2 3 4 5 6 7 8	

BÀI 1.2.23: SẮP XẾP CHÈN

Hãy thực hiện thuật toán sắp xếp chèn trên dãy N số nguyên. Ghi ra các bước thực hiện thuật toán.

Dữ liệu vào: Dòng 1 ghi số N (không quá 100). Dòng 2 ghi N số nguyên dương (không quá 100).

Kết quả: Ghi ra màn hình từng bước thực hiện thuật toán. Mỗi bước trên một dòng, các số trong dãy cách nhau đúng một khoảng trống.

Ví dụ:

Input	Output
4	Buoc 0: 5
5 7 3 2	Buoc 1: 5 7